

# 2023-01-31 미니프로젝트3

MMDetection 복습을 위해 COCO Dataset을 이용한 미니프로젝트3 진행

## [Train]

```
train_tool_animal.py > ...
1  import torch, mmdcv
2  from mmdcv import Config
3  from mmdet.apis import init_detector, inference_detector, set_random_seed, train_detector
4  from mmdet.datasets.builder import DATASETS
5  from mmdet.datasets.coco import CocoDataset
6  from mmdet.datasets import build_dataset
7  from mmdet.models import build_detector
8  from mmdet.runner import get_dist_info, init_dist
9  from mmdet.utils import collect_env, get_root_logger, setup_multi_processes
```

필요한 라이브러리 불러오기

```
12  """Dataset Register"""
13  @DATASETS.register_module(force=True) # 강제로 클래스 정보 수정할 거기 때문에 넣음
14  class AnimalDataset(CocoDataset): # 상속받는 건 CocoDataset | json 파일에 적힌 label(=class 정보)대로 입력하면 됨
15      # mmdet\datasets\coco.py
16      CLASSES = ('cat', 'chicken', 'cow', 'dog', 'fox', 'goat', 'horse', 'person', 'raccoon', 'skunk')
17
18
19  """Config file"""
20  # cascade_rcnn(겁나 무거움), dynamic_rcnn, fast_rcnn, mask_rcnn(segmentation 할 때 씬) | 애네를 좀 많이 씬
21  config_file = './configs/dynamic_rcnn/dynamic_rcnn_r50_fpn_1x_coco.py' # r50은 resnet50을 의미함
22  cfg = Config.fromfile(config_file) # 파일 읽어들이기
23  # print(cfg.pretty_text) # 해당 config file의 아키텍처 구조나 정보를 보고 싶을 때 사용
24
25
26  """Learning Rate setting"""
27  # single GPU default --> 0.0025
28  cfg.optimizer.lr = 0.0025 # 0.02 / 8 : 1개의 GPU로 하는 거라서 8로 나눠줌. 원래 학습을 8개 GPU로 하기 때문
```

데이터셋을 등록하고, json파일에 적힌 클래스 정보, Config파일에 대한 선언을 해줌 + learning rate 선언해줌(GPU 개수가 1개이기에 0.0025로 표기)

```
"""Dataset setting"""
cfg.dataset_type = 'AnimalDataset' # configs\_base\_datasets\coco_detection.py 참고
cfg.data_root = './animals.v2-release.coco'

"""Train, Val, Test dataset의 type, data root, annotation file, img_prefix 설정"""
cfg.data.train.type = 'AnimalDataset' # train
cfg.data.train.ann_file = './animals.v2-release.coco/train/_annotations.coco.json' # 이 파일을 읽고 처리해줘!
cfg.data.train.img_prefix = './animals.v2-release.coco/train/' # 뒤에 알아서 train 붙을 거기 때문에 여기까지만 써주면 됨
cfg.data.val.type = 'AnimalDataset' # valid
cfg.data.val.ann_file = './animals.v2-release.coco/valid/_annotations.coco.json'
cfg.data.val.img_prefix = './animals.v2-release.coco/valid/'
cfg.data.test.type = 'AnimalDataset' # test
cfg.data.test.ann_file = './animals.v2-release.coco/test/_annotations.coco.json'
cfg.data.test.img_prefix = './animals.v2-release.coco/test/'

cfg.model.roi_head.bbox_head.num_classes = 10 # class number 수정하기 | models쪽 타고 넘어감
cfg.model.rpn_head.anchor_generator.scales = [4] # 작은 객체를 잡기 위해 anchor size를 변경할 것임 | default=8 --> 4
cfg.load_from = './dynamic_rcnn_r50_fpn_1x-62a3f276.pth' # pretrained-model pt파일 불러오기, 다운로드 받아서 넣어야 함
cfg.work_dir = './work_dirs/0131_animal' # 학습한 모델 저장 경로 지정

cfg.lr_config.warmup = None # learning rate hyperparameter 설정
cfg.log_config.interval = 10 # 10번째마다 보여줘
```

- 데이터셋을 위에서 등록했던 것으로 선언해주고 root 경로 위치, train-valid-test 데이터셋의 type, annotation file(json), 이미지 데이터 경로 지정

- Bbox를 잡고 클래스 개수를 맞춰주기 위해 선언. 작은 객체를 잡기 위해 anchor는 4로 설정
- 사전 학습된 모델을 다운로드 받아서 불러오고, 저장할 경로 지정, 저장할 interval 설정

```

55 '''[평가]COCO dataset evaluation type = bbox 지정''' # mAP로 놓는 경우도 있음
56 # mAP IoU threshold 0.5 ~ 0.95 변경하면서 측정
57 cfg.evaluation.metric = 'bbox'
58 cfg.evaluation.interval = 10 # 평가 몇 번에 한 번씩 할래? 보통 10번 정도마다 봄
59 cfg.checkpoint_config.interval = 88 # 모델 파일 중간 저장 몇 번에 한 번씩 할래?
60
61 cfg.runner.max_epochs = 88 # epoch 설정 | 기본값 : 8(GPU 갯수) x 12 = 96, 지금은 GPU 1개라 저렇게 설정
62 cfg.seed = 777 # 고정된 seed값 지정
63 cfg.data.samples_per_gpu = 8 # single GPU에서 통용되는 값 | batch_size라고 이해하면 됨
64 cfg.data.workers_per_gpu = 2 # num_workers, single GPU에서 통용되는 값 | 애네는 거의 고정값
65 # print('cfg.data >>', cfg.data)
66
67 cfg.gpu_ids = range(1) # GPU 사용 갯수
68 cfg.device = 'cuda' # device 지정
69 set_random_seed(777, deterministic=False) # 고정된 seed값 지정
70 # print('cfg info show >>> ', cfg.pretty_text)
71
72 datasets = [build_dataset(cfg.data.train)] # train용 데이터를 생성하고 올리자
73 # print('datasets[0]', datasets[0])
74
75 datasets[0].__dict__.keys() # datasets[0].__dict__ variables key val
76
77 model = build_detector(cfg.model, train_cfg=cfg.get('train_cfg'), test_cfg=cfg.get('test_cfg'))
78 model.CLASSES = datasets[0].CLASSES
79 # print(model.CLASSES)
80
81 if __name__ == '__main__': # 애는 num_worker로 인한 이유인지 이런 방식으로 실행해야 함. ipynb에서 할거면 조건문 없이 실행
82     train_detector(model, datasets, cfg, distributed=False, validate=True)

```

중간마다 점수로 평가를 하기 위해 config파일 설정

- 잡을 단위(?)는 bbox. Interval 횟수는 10회마다로 설정. 중간에 모델 파일의 용량으로 인한 스토리지 압박으로 인해 최종 파일만 저장하도록 모델 저장 interval은 max\_epoch 횟수로 선언 하였음
- 이후 데이터셋 생성하고 조건문으로 실행

## [Inference]

```

Inference_animal.py > ...
1 import cv2, json, os, glob, numpy as np
2 from mmdet.apis import inference_detector, init_detector, set_random_seed
3 from mmdet.datasets import build_dataset, DATASETS
4 from mmdet.models import build_detector
5 from mmdet.datasets.coco import CocoDataset
6 from mmcv import Config
7 """COCO Dataset 기준으로 봤을 때 기본 구조"""
8
9 """Config 설정(train과 동일함)"""
10
11 """Dynamic RCNN 설정"""
12 config_file = './configs/dynamic_rcnn/dynamic_rcnn_r50_fpn_1x_coco.py'
13 cfg = Config.fromfile(config_file) # cfg라는 변수 선언해주고 꼭 다뤄나간다
14
15
16 """DATASETS.register_module로 등록"""
17 @DATASETS.register_module(force=True)
18 class WineLabelsDataset(CocoDataset): # 상속받는 건 CocoDataset | json 파일에 적힌 label(class 정보)대로 입력하면 됨
19     # mmdet\datasets\coco.py | 번호로 들어가기 때문에 한글로 클래스 입력해도 동작한다
20     CLASSES = ('cat', 'chicken', 'cow', 'dog', 'fox', 'goat', 'horse', 'person', 'raccoon', 'skunk')
21
22
23 """dataset 설정"""
24 cfg.dataset_type = 'AnimalDataset' # configs\_base\_datasets\coco_detection.py 참고
25 cfg.data_root = './animals.v2-release.coco'

```

Config파일은 Train에서 작성했던 것과 거의 유사하게 작성함

- Config 파일 선언해서 수정하는 점, 데이터셋 등록, 클래스 정보 입력, 데이터셋 경로 지정

```

27
28 """Train, Valid and Test dataset의 type, data_root, ann_file, img_prefix 등 하이퍼파라미터 지정"""
29 cfg.data.train.type = 'AnimalDataset' # train
30 cfg.data.train.ann_file = './animals.v2-release.coco/train/_annotations.coco.json'
31 cfg.data.train.img_prefix = './animals.v2-release.coco/train/' # 뒤에 알아서 train 붙을 거기 때문에 여기까지만 써주면 됨
32 cfg.data.val.type = 'AnimalDataset' # valid
33 cfg.data.val.ann_file = './animals.v2-release.coco/valid/_annotations.coco.json'
34 cfg.data.val.img_prefix = './animals.v2-release.coco/valid/'
35 cfg.data.test.type = 'AnimalDataset' # test
36 cfg.data.test.ann_file = './animals.v2-release.coco/test/_annotations.coco.json'
37 cfg.data.test.img_prefix = './animals.v2-release.coco/test/'
38
39 cfg.model.roi_head.bbox_head.num_classes = 10 # 모델에 들어가는 클래스 갯수 수정. resnet 계열이 대체로 roi_head 부분에서 설정할
40 cfg.load_from = './dynamic_rcnn_r50_fpn_1x-62a3f276.pth' # Pretrained model의 아키텍처 불러오기
41 cfg.work_dir = './0131' # weight file save directory 설정
42
43 cfg.lr_config.warmup = None # train 하이퍼파라미터 설정
44 cfg.log_config.interval = 10

```

평가에 사용할 데이터셋들의 각종 하이퍼파라미터 및 평가에 사용할 수정할 클래스 개수, 사전 학습된 모델의 아키텍처 선언

```

47 """[평가]COCO dataset evaluation type = bbox 지정"""
48 # VOC 데이터 같은 경우에는 mAP로 놓는 경우도 있음
49 # mAP IoU threshold 0.5 ~ 0.95 변경하면서 측정
50 cfg.evaluation.metric = 'bbox'
51 cfg.evaluation.interval = 10
52 cfg.checkpoint_config.interval = 10
53
54 cfg.runner.max_epochs = 10 # epoch 설정. test라서 10회 정도로만 설정
55 cfg.seed = 0 # 고정된 seed값 지정
56 cfg.gpu_ids = range(1)
57 set_random_seed(0, deterministic=False)
58
59 checkpoint_file = './work_dirs/0131_animal/latest.pth' # 평가에 사용할(=학습에 사용했던) 모델을 불러오자
60 model = init_detector(cfg, checkpoint_file, device='cuda') # train에 사용했던 정보로 해야한다
61

```

평가를 위해 타입을 bbox로 지정해주고 test이기에 epoch 횟수는 10회로 선언

앞서 train을 통해 생성된 모델을 불러오고 train에서 사용했던 정보대로 init\_detector 선언

```

70 """json파일에서 이미지 이름과 각종 정보들을 뽑아내자"""
71 img_info_path = './animals.v2-release.coco/test/_annotations.coco.json'
72 with open(img_info_path, 'r', encoding='utf-8') as f :
73     image_info = json.loads(f.read())
74 # print(image_info)
75
76 """임계값 설정"""
77 score_threshold = 0.7
78
79
80 """JSON을 만들기 위해 빈 리스트 선언"""
81 submission_anno = list()

```

평가 후에 생성할 JSON 파일 생성을 위한 빈 리스트 선언, 임계값 설정, annotation 파일에서 이미지의 각종 정보를 뽑아내기 위해 with open으로 읽어옴

```

83 """이미지를 한 개씩 순회하면서 inference를 보는 구간"""
84 for img_info in image_info['images'] : # json 데이터 상위에 위치
85     # print(img_info) # {'id': 629, 'license': 1, 'file_name': 'tjwcg1wtrukphohufrun_jpg.rf.f0ce477d819dfff197254ed8ed070923.jpg'}
86
87     file_name = img_info['file_name'] # json - images - file_name
88     img_width = img_info['width'] # json - images - width
89     img_height = img_info['height'] # json - images - height
90     # print(file_name, img_width, img_height) # kcx3xqc9cxd61axefwgv_jpg.rf.03a85546c8ffe0a266ed771579c77429.jpg 480 310
91
92     img_path = os.path.join('./animals.v2-release.coco/test/', file_name) # 입력한 경로 안의 파일을 보자
93     # print(img_path) # ./dataset/test/kcx3xqc9cxd61axefwgv_jpg.rf.03a85546c8ffe0a266ed771579c77429.jpg
94
95     image = cv2.imread(img_path)
96     image_copy = image.copy() # 원본이랑 비교해보게 복사를 해두자
97     image_resize = cv2.resize(image_copy, (960, 540)) # 위에서 복사해둔 걸 리사이즈
98
99     '''scale 구하기'''
100     x_scale = float(960) / img_width
101     y_scale = float(540) / img_height
102     # print(x_scale, y_scale) # 1.125 3.096774193548387
103

```

이미지를 한 개씩 순회해가며 inference를 보기 위해 json에서 이미지에 대한 정보를 뽑아옴. 이를 cv2로 읽고 복사해서 하단에서 열람하기 위한 용도. 리사이즈를 위해선 조정되는 비율(scale)을 알기 위해 이를 위한 계산 필요

```

104 results = inference_detector(model, img_path) # 이미지 경로로 넣어도 동작함(내부적으로 처리가 되어 있는 듯)
105 for number, result in enumerate(results) :
106     if len(result) == 0 : # 이미지 내용(?)이 없다면(=object detect된 게 없으면)
107         continue # 그냥 진행
108     category_id = number + 1
109     result_filtered = result[np.where(result[:, 4] > score_threshold)] # threshold 설정 : score_threshold보다 높은 것(=boundi
110     if len(result_filtered) == 0 : # bounding box 갇힌 게 없다면?
111         continue # GPU 아까우니 종료해라 or 다음 꺼로 넘어가라
112
113     for i in range(len(result_filtered)) : # 0.5 이상으로 살아남은 애들에 대해서는 하나씩 좌표를 뽑아보자
114         # print(result_filtered) # [[], []] --> [0][0], [0][1] 이종으로 된 list임
115
116         tmp_dict = dict() # json은 dict 형태라 하나 만들어주자
117         x_min = int(result_filtered[i, 0])
118         y_min = int(result_filtered[i, 1])
119         x_max = int(result_filtered[i, 2])
120         y_max = int(result_filtered[i, 3])
121         # print(x_min, y_min, x_max, y_max)
122
123         '''xyxy --> xywh로 변환'''
124         json_x = x_min
125         json_y = y_min
126         json_w = x_max - x_min
127         json_h = y_max - y_min
128         # print(json_x, json_y, json_w, json_h).
129

```

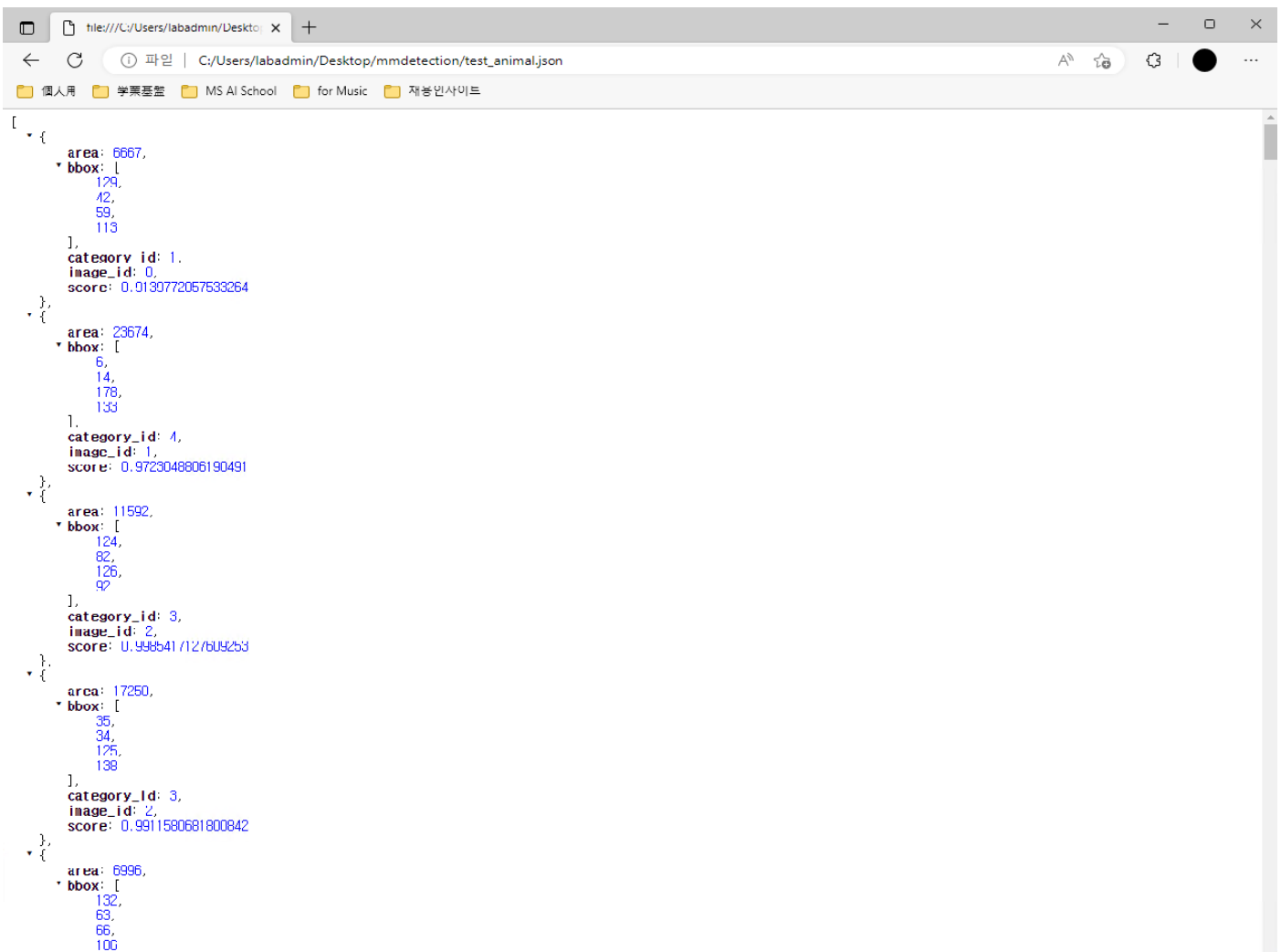
Inference\_detector에 넣고 나온 결과(results)를 선언해주고 여기서 임계값보다 어떤지를 조건문을 걸어서 필터링을 해주고, 살아남은 이미지들 및 anno 정보 대상으로 필요로 하는 값을 연산해준다. Json에 기록하기 위해서 xyxy 형식을 xywh 형식으로 변환해주는 연산 또한 해줌

```

129
130     '''JSON 정보를 작성'''
131     tmp_dict['bbox'] = [json_x, json_y, json_w, json_h]
132     tmp_dict['category_id'] = category_id
133     tmp_dict['area'] = json_w * json_h # 해당하는 box의 면적 --> 너무 작으면 학습해도 의미가 없어서 제외하는 용도
134     tmp_dict['image_id'] = img_info['id']
135     tmp_dict['score'] = float(result_filtered[i, 4])
136     submission_anno.append(tmp_dict)
137
138     '''scale bbox - 해당하는 scale(비율)로 변환'''
139     x1 = int(x_min * x_scale)
140     y1 = int(y_min * y_scale)
141     x2 = int(x_max * x_scale)
142     y2 = int(y_max * y_scale)
143     cv2.rectangle(image_resize, (x1, y1), (x2, y2), (0, 255, 0), 2)
144     # cv2.rectangle(image, (x_min, y_min), (x_max, y_max), (0, 255, 255), 2)
145     cv2.imshow('test', image_resize)
146     # cv2.imshow('test', image)
147     if cv2.waitKey() == ord('q') : # q 입력하면 종료
148         exit()
149
150     '''위에서 작성한 정보들로 json 파일을 작성해서 생성'''
151     with open('./test_animal.json', 'w', encoding='utf-8') as f :
152         json.dump(submission_anno, f, indent=4, sort_keys=True, ensure_ascii=False)

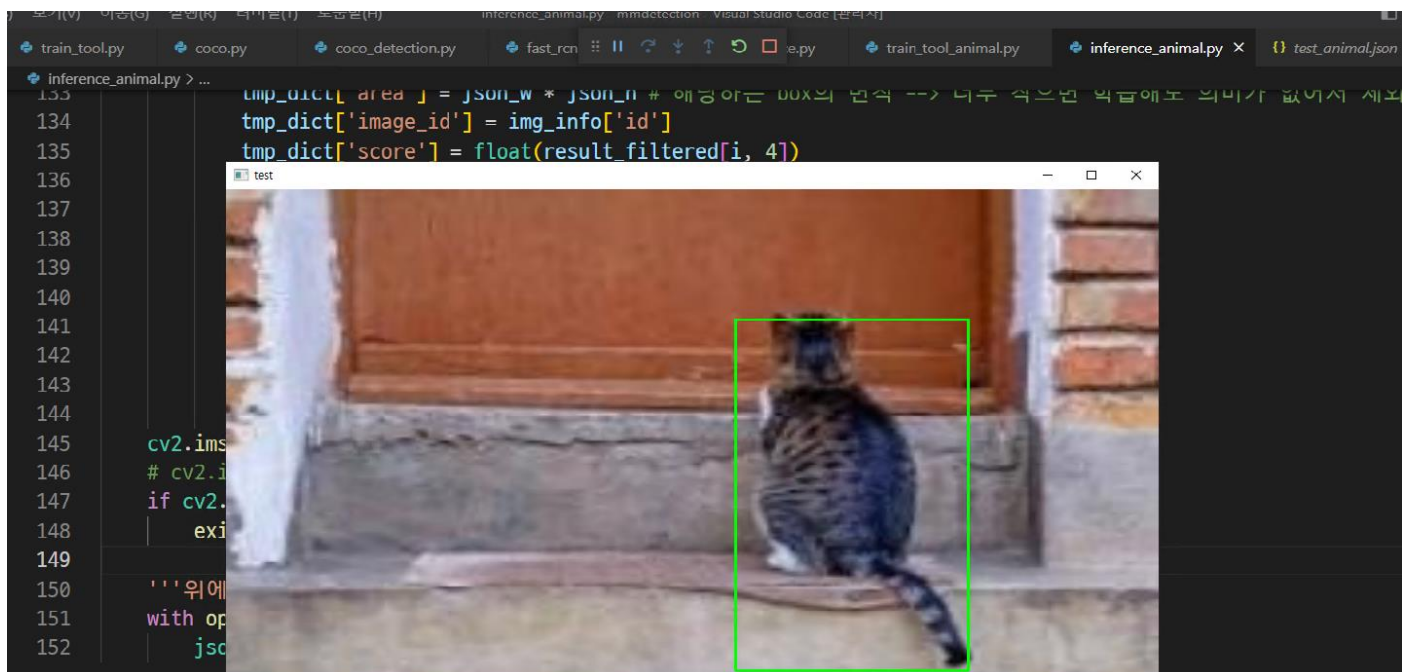
```

- JSON 정보 작성을 해준다. 위에서 선언한 tmp\_dict에 필요로 하는 정보를 키값으로 해서 넣어 주고, 이를 마지막에 다시 리스트에 추가해서 완성시킨다. Inference를 거친 이미지를 열람하기 위해 앞서 복사+리사이즈했던 이미지를 scale(비율)에 맞춰 변환해준 뒤 이를 표기하게 함
- 위에서 작성하였던 정보들로 JSON 파일을 dump를 통해 생성해줌



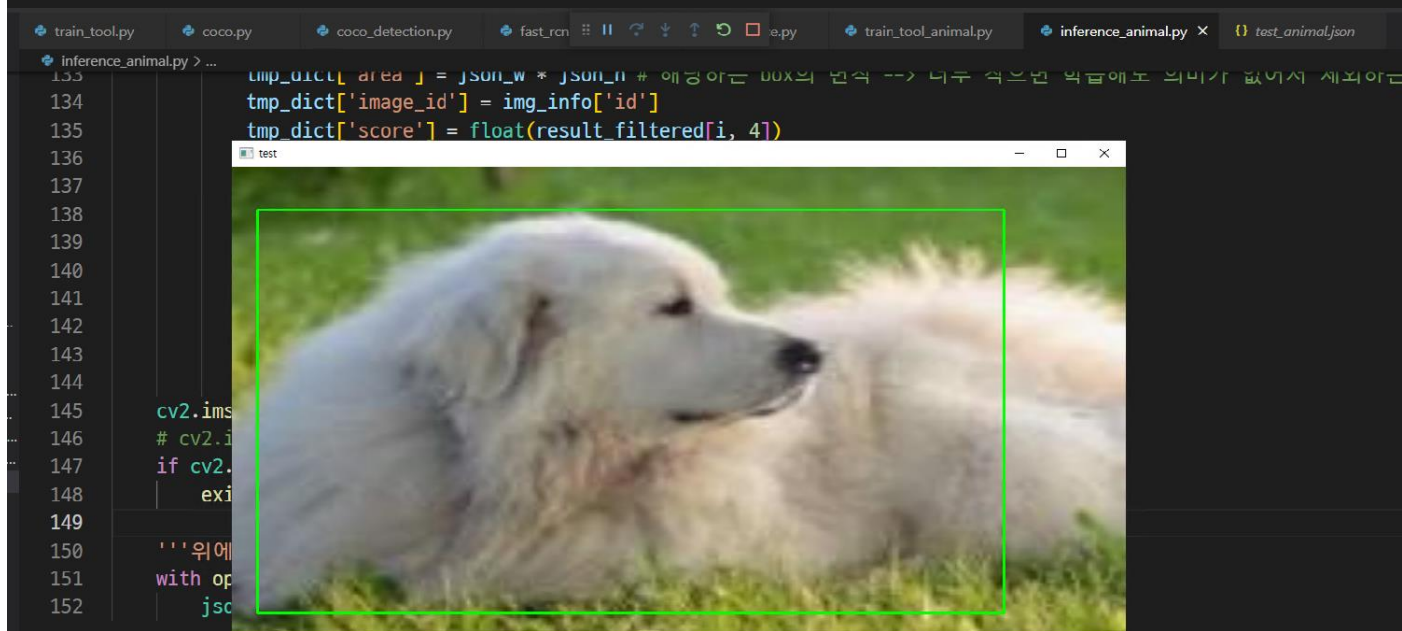
생성된 JSON파일(test\_animal.json)





문제 출력 디버그 콘솔 더미널

```
(mmdetection) C:\Users\labadmin\Desktop\mmdetection> c: && cd c:\Users\labadmin\Desktop\mmdetection && cmd /C "C:\Users\labadmin\miniconda3\envs\mmdetection\python.exe c:\Users\labadmin\vscode\extensions\ms-python.python-2022.20.2\pythonFiles\lib\python\wdebugpy\wadapter\...\wdebugpy\launcher 55778 -- c:\Users\labadmin\Desktop\mmdetection\inference_animal.py"
load checkpoint from local path: ./work_dirs/0101_animal/latest.pth
c:\Users\labadmin\Desktop\mmdetection\mmdet\utils.py:66: UserWarning: "ImageLolensor" pipeline is replaced by "DefaultFormatBundle" for batch inference. It is recommended to manually replace it in the test data pipeline in your config file.
  warnings.warn()
```



문제 출력 디버그 콘솔 더미널

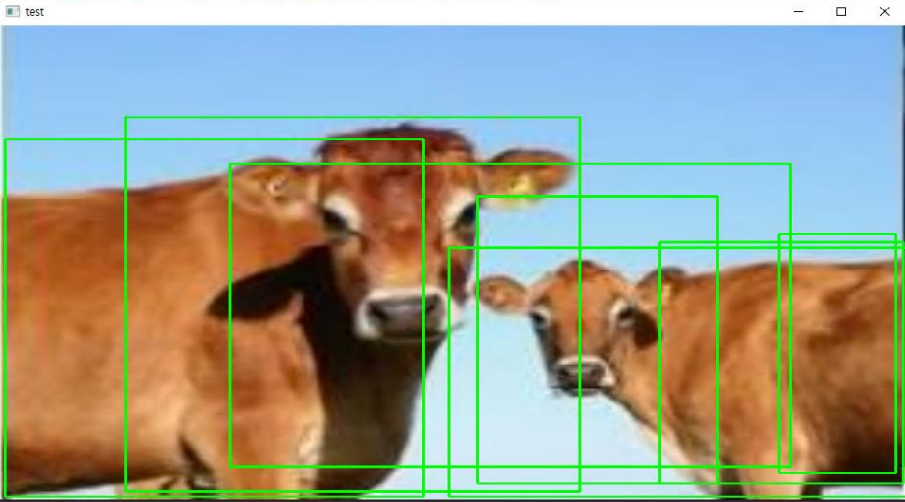
```
(mmdetection) C:\Users\labadmin\Desktop\mmdetection> c: && cd c:\Users\labadmin\Desktop\mmdetection && cmd /C "C:\Users\labadmin\miniconda3\envs\mmdetection\python.exe c:\Users\labadmin\vscode\extensions\ms-python.python-2022.20.2\pythonFiles\lib\python\wdebugpy\wadapter\...\wdebugpy\launcher 55778 -- c:\Users\labadmin\Desktop\mmdetection\inference_animal.py"
load checkpoint from local path: ./work_dirs/0101_animal/latest.pth
c:\Users\labadmin\Desktop\mmdetection\mmdet\utils.py:66: UserWarning: "ImageLolensor" pipeline is replaced by "DefaultFormatBundle" for batch inference. It is recommended to manually replace it in the test data pipeline in your config file.
  warnings.warn()
```

train\_tool.py coco.py coco\_detection.py fast\_rcn inference\_animal.py test\_animal.json

```

inference_animal.py > ...
133 tmp_dict['area'] = json_w * json_h # 애강아는 00x의 면적 --> 너무 작은 박스에도 의미가 없어서 제외
134 tmp_dict['image_id'] = img_info['id']
135 tmp_dict['score'] = float(result_filtered[i, 4])
136
137
138
139
140
141
142
143
144
145 cv2.imshow('test', img)
146 # cv2.waitKey(0)
147 if cv2.waitKey(1) & 0xFF == ord('q'):
148     exit()
149
150 '''위에
151 with op
152 jsc

```



문제 출력 디버그 콘솔 터미널

```

(mmdetection) C:\Users\labadmin\Desktop\mmdetection> c: && cd c:\Users\labadmin\Desktop\mmdetection && cmd /C "C:\Users\labadmin\miniconda3\envs\mmdetection\python.exe c:\Users\labadmin\vscode\extensions\ms-python.python-2022.20.2\pythonFiles\lib\python\debugpy\adapter\..\..\debugpy\launcher 55778 -- c:\Users\labadmin\Desktop\mmdetection\inference_animal.py"
load checkpoint from local path: ./work_dirs/0101_animal/latest.pth
c:\Users\labadmin\Desktop\mmdetection\mmdet\datasets\utils.py:66: UserWarning: "ImageLentensor" pipeline is replaced by "DefaultFormatBundle" for batch inference. It is recommended to manually replace it in the test data pipeline in your config file.
  warnings.warn(
[]

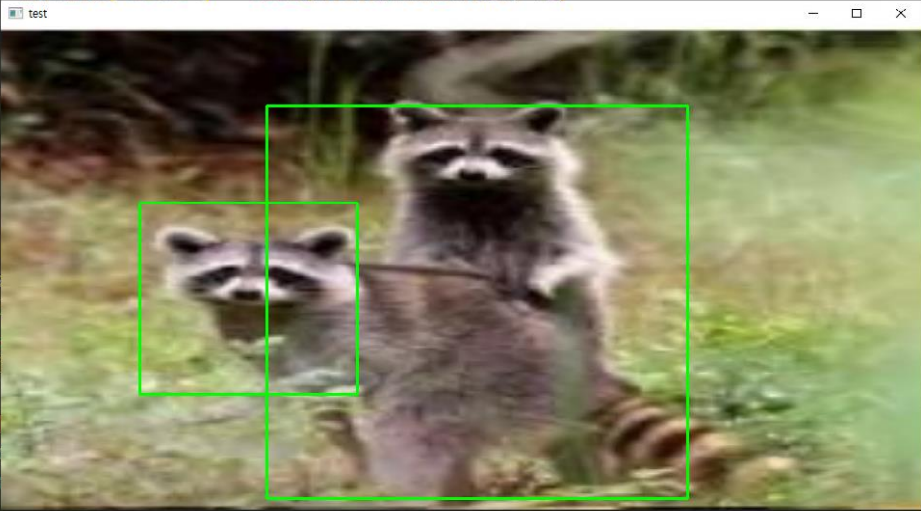
```

train\_tool.py coco.py coco\_detection.py fast\_rcn inference\_animal.py test\_animal.json

```

inference_animal.py > ...
133 tmp_dict['area'] = json_w * json_h # 애강아는 00x의 면적 --> 너무 작은 박스에도 의미가 없어서 제외
134 tmp_dict['image_id'] = img_info['id']
135 tmp_dict['score'] = float(result_filtered[i, 4])
136
137
138
139
140
141
142
143
144
145 cv2.imshow('test', img)
146 # cv2.waitKey(0)
147 if cv2.waitKey(1) & 0xFF == ord('q'):
148     exit()
149
150 '''위에
151 with op
152 jsc

```



문제 출력 디버그 콘솔 터미널

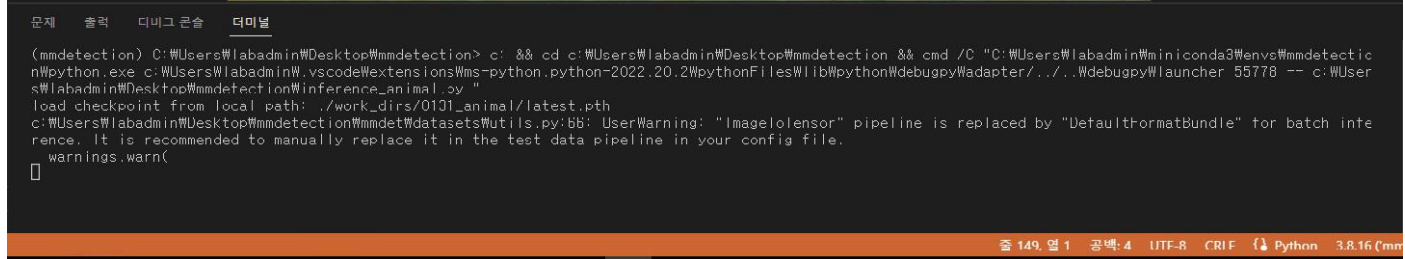
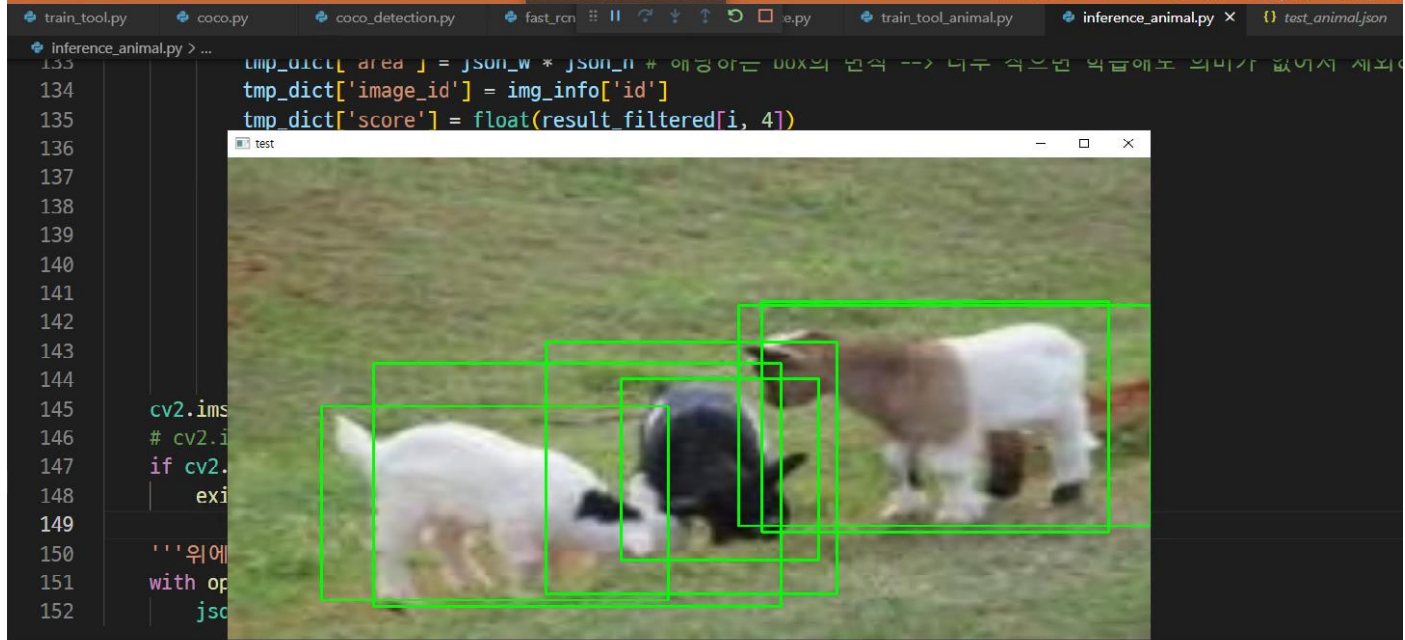
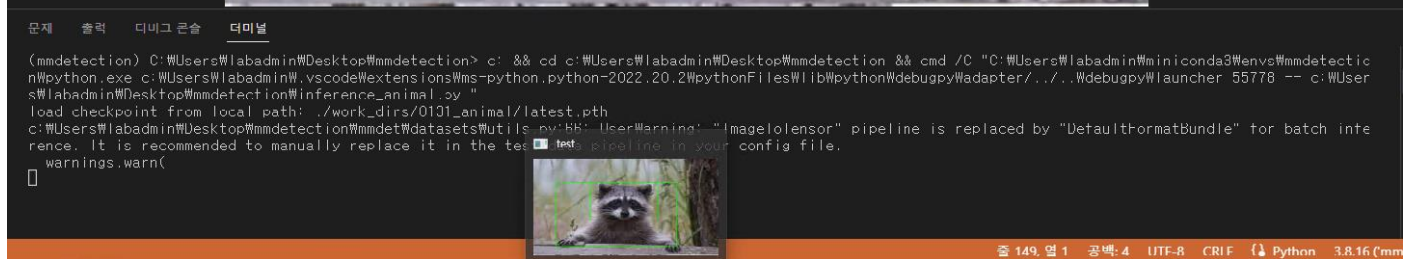
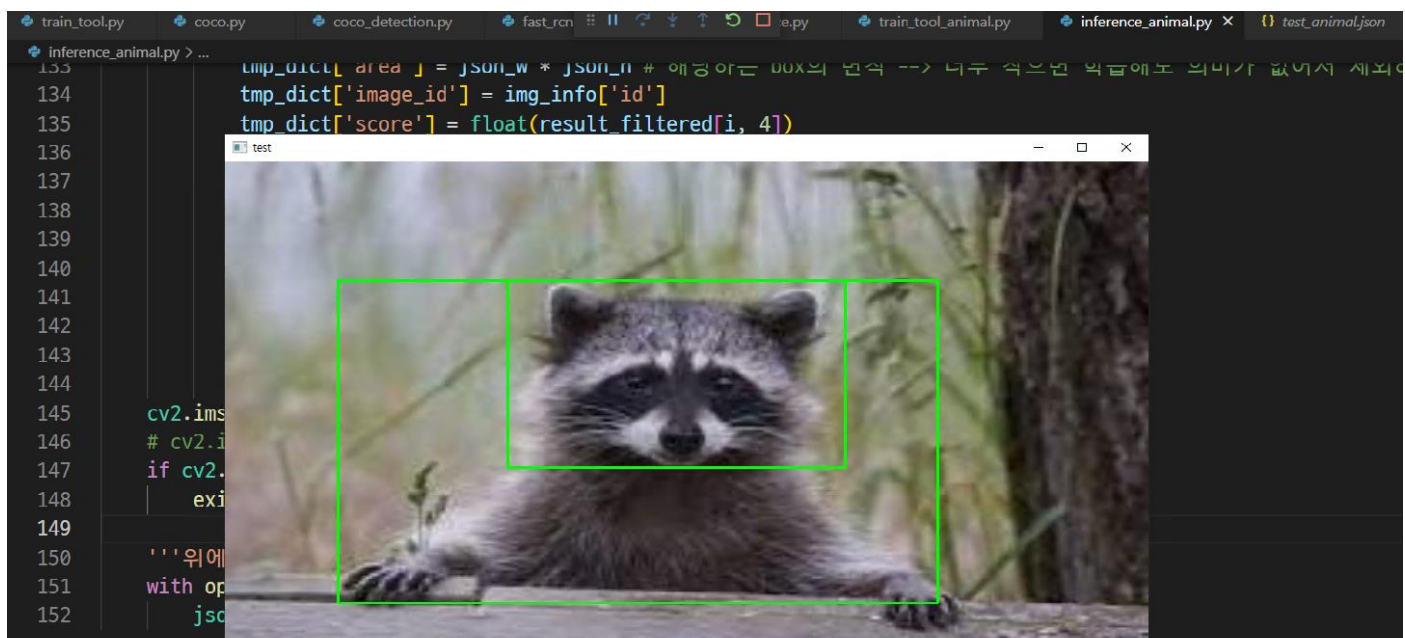
```

(mmdetection) C:\Users\labadmin\Desktop\mmdetection> c: && cd c:\Users\labadmin\Desktop\mmdetection && cmd /C "C:\Users\labadmin\miniconda3\envs\mmdetection\python.exe c:\Users\labadmin\vscode\extensions\ms-python.python-2022.20.2\pythonFiles\lib\python\debugpy\adapter\..\..\debugpy\launcher 55778 -- c:\Users\labadmin\Desktop\mmdetection\inference_animal.py"
load checkpoint from local path: ./work_dirs/0101_animal/latest.pth
c:\Users\labadmin\Desktop\mmdetection\mmdet\datasets\utils.py:66: UserWarning: "ImageLentensor" pipeline is replaced by "DefaultFormatBundle" for batch inference. It is recommended to manually replace it in the test data pipeline in your config file.
  warnings.warn(
[]

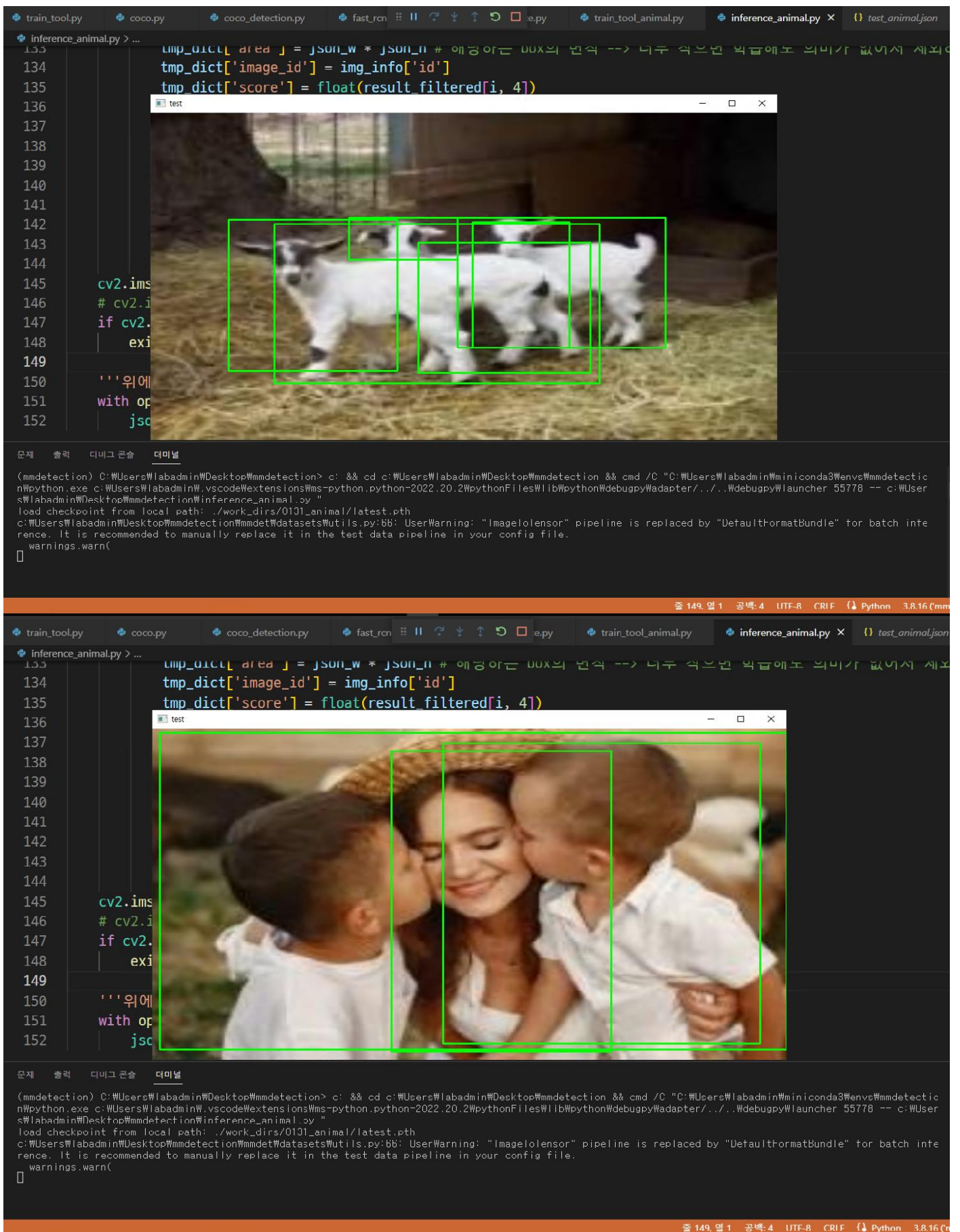
```

줄 149, 열 1 공백: 4 UTF-8 CR LF Python 3.8.16 (mmdetection)









Inference를 거친 이미지들 - 클래스 정보에 맞게 bbox를 잘 잡는 경우도 있었으나 bbox를 너무 넓게 잡거나, 객체 수보다 훨씬 많이 잡거나, 크기나 범위에 맞지 않게 잡거나 하는 경우가 상당히 있었음. Threshold를 0.7로 두었음에도 만족스럽지 않은 결과. 클래스가 10개인데 학습 데이터가 700장에 불과해 학습에 필요한 데이터량이 부족한 것으로 판단함.